

散列表 Hash Tables

1.Direct-address Tables

分配一个大小为 $m = |U|$ 的数组, 其中 $U = \{0, 1, 2, \dots, m-1\}$.

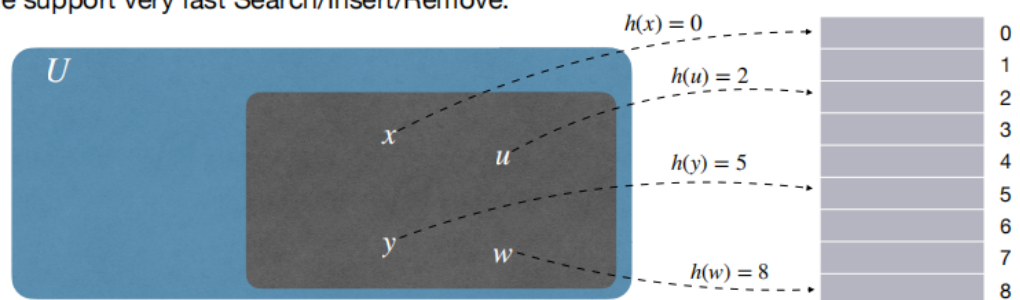
*Search/Insert/Remove*操作可以在 $O(1)$ 时间内.

但是实际问题中 U 往往很大, 如何解决空间复杂度?

2.Hashing

- Huge universe U of possible keys.
- Much smaller number n of actual keys.
- Only want to spend $m \approx n$ (i.e., $m \leq |U|$) space.
- Hash function $h:U \rightarrow [m], h(k)$ decides index of slot for storing key k .

► Meanwhile support very fast Search/Insert/Remove.

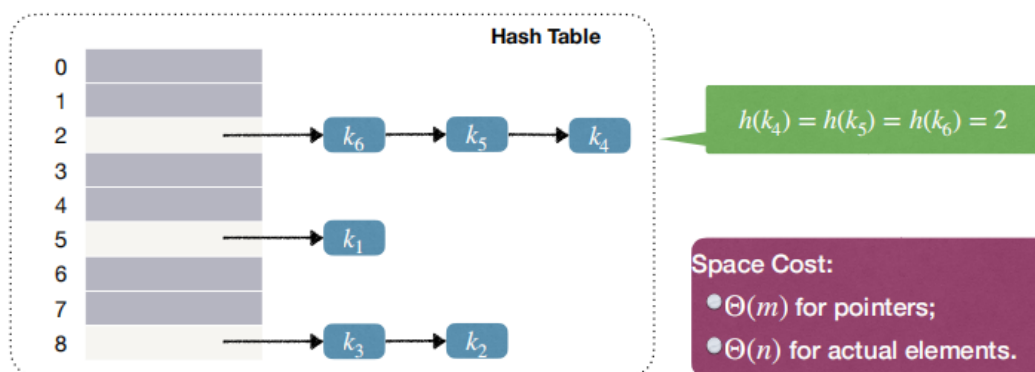


2.1 Collisions in hashing

- 哈希函数 $h:U \rightarrow [m]$
- 当 $h(k_1) = h(k_2)$ 时两个不同的键值 k_1, k_2 产生矛盾

2.2 Chaining--cope with collisions in hashing

- 每个桶 i 都存储一个指向链表 L_i 的指针
- 所有被散列到索引 i 的键都将转到链表 L_i



Space Cost:

- $\Theta(m)$ for pointers;
- $\Theta(n)$ for actual elements.

2.3 Hashing with Chaining

- *Search*(k), k 是键值
 - 计算 $h(k)$
 - 通过相应的列表来搜索带有键值 k 的项。

时间取决于链表的长度! 最坏的情况要花费 $\Theta(n)$ (所有的键值都被散列到相同的值)

- *Insert*(x), x 是指向其中一个键值的指针
 - 计算 $h(x.key)$
 - 把 x 插入到相应的列表的头部

时间复杂度为 $O(1)$

- *Remove*(x), x 是指向其中一个键值的指针
 - 只需把 x 从链表中移除

时间复杂度为 $O(1)$

2.4 Simple Uniform Hashing Assumption(简单统一散列)

给定一个哈希表 T ，其中有 m 个插槽，存储了 n 个元素，我们将 T 的负载因子定义为 $\alpha = \frac{n}{m}$ ，即链中存储的平均元素数。我们的分析将包括：也就是说，我们想象当 n 和 m 走向无穷大时保持固定。（请注意， α 可以小于、等于或大于1）。

我们假设哈希值 $h(k)$ 可以在 $O(1)$ 时间内计算，因此搜索具有键 k 的元素所需的时间线性取决于列表 $T[h(k)]$ 的长度。撇开计算哈希函数和访问槽 $h(k)$ 所需的 $O(1)$ 时间不谈，让我们考虑搜索算法检查的预期元素数，即列表 $T[h(k)]$ 中检查其键是否等于 k 的元素数。我们将考虑两种情况。在第一种情况下，搜索不成功：表中没有任何元素具有键 k 。在第二种情况下，搜索成功找到了键为 k 的元素。

搜索平均花费的时间为 $O(1 + \alpha)$

- $O(1)$ 用来计算哈希值
- $O(\alpha)$ 用来遍历链表

不成功搜索: $\Theta(1 + \alpha)$

成功搜索: $\Theta(1 + \alpha)$

- 令 C_i 为找到第 i 个插入元素 x_i 的代价，我们需要计算 $\frac{1}{n} \cdot \sum_{i=1}^n E[C_i]$.
- 令 X_{ij} 为一个指标随机变量，当且仅当 $h(x_i, key) = h(x_j, key)$ 时取值为1.
- 则 $\frac{1}{n} \cdot \sum_{i=1}^n E[C_i] = \frac{1}{n} \cdot \sum_{i=1}^n E[(1 + \sum_{j=i+1}^n X_{ij})]$
- $= \frac{1}{n} \cdot \sum_{i=1}^n (1 + \sum_{j=i+1}^n E[X_{ij}]) = \frac{1}{n} \cdot \sum_{i=1}^n (1 + \sum_{j=i+1}^n \frac{1}{m})$
- $= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} = \Theta(1 + \alpha)$

如果 $m = \Theta(n)$, 哈希表花费 $\Theta(n)$ 空间，但是 *Search/Insert/Remove* 操作全部平均只需 $O(1)$ 的时间。

3. Design hash functions--设计哈希函数

3.1 Some bad hash functions--一些坏的哈希函数

假设键值为英文字母，一个桶中一个字母，也就是有26个桶。哈希函数: $h(w) = w$ 的第一个字母。这样的话有的字母开头的单词多，有的少。

3.2 The Division Method--除法哈希

哈希函数: $h(k) = k \bmod m$, 例如: 如果 $m = 13, h(24) = 11$ 。

- 如何选择 m ? m 不应该为2的幂， m 的良好值是不太接近2的精确幂的素数。

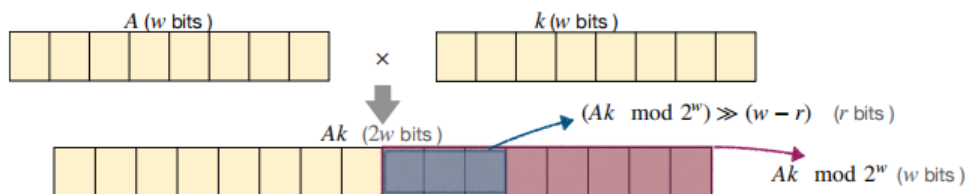
3.3 The Multiplication Method--乘法哈希

哈希函数: $h(k) = \lfloor m(kA \bmod 1) \rfloor, 0 < A < 1$

其中 $kA \bmod 1$ 表示 kA 的小数部分，即 $kA - \lfloor kA \rfloor$

- Another common technique when designing hash functions

- Assume key length is at most w bits.
- Fix table size $m = 2^r$ for some $r \leq w$.
- Fix constant integer $0 < A < 2^w$.
- Hash function: $h(k) = (Ak \bmod 2^w) \gg (w - r)$



乘法哈希比除法哈希快，但是：一旦哈希函数被固定并已知，就必须存在一组“坏”键，它们哈希到相同的值。这种对抗性的输入会导致性能不佳！

3.4 Universal Hashing--通用哈希

在第一次构建哈希表时，选择一个随机的哈希函数 h ，一旦被选择， h 在整个执行过程中都会被固定。

在以下情况下，哈希函数的集合 $\mathcal{H}$ 是通用的：

- 对于任何 $x \neq y$ ，最多 $\frac{|\mathcal{H}|}{m}$ 个哈希函数使得 $h(x) = h(y)$ 。
- 因此，对于所有 $x \neq y$ ，有 $Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$ 。

简单统一假设和通用哈希的比较：

- 简单统一哈希：输入的随机性导致的不确定性
- 通用哈希：哈希函数的选择的随机性（以及潜在的输入的随机性）导致的不确定性

令 $L_{h(k)}$ 为索引为 $h(k)$ 的链表的长度

- Claim 1: 如果键值 k 不在表中，则 $E[L_{h(k)}] \leq \alpha$
 - 对于任意键值 l ，定义指标随机变量 $X_{kl} = I\{h(k) = h(l)\}$
 - 则： $E[L_{h(k)}] = E[\sum_{l \in T, l \neq k} X_{kl}] = \sum_{l \in T, l \neq k} E[X_{kl}] \leq n \cdot \frac{1}{m} = \alpha$
- Claim 2: 如果键值 k 在表中，则 $E[L_{h(k)}] \leq 1 + \alpha$
 - $E[L_{h(k)}] = E[\sum_{l \in T, l \neq k} X_{kl}] + 1 \leq (n - 1) \cdot \frac{1}{m} + 1 = 1 + \alpha$

3.5 A typical universal hash family--通用哈希族

- 散列的概念很简单：一个未知的集合 S 之 n 数据项（或键）是从大宇宙中提取的 $U = [N]$ 哪里 $N \gg n$ ；为了存储 S 在表格中 M 条目（插槽），我们假设来自宇宙的一致映射（称为**哈希函数**） U 到小范围 $[M]$ 。
- 这个想法看起来很聪明：我们使用一致的映射来处理任意未知数据集。但是，哈希存在一个根本缺陷。对于足够大的宇宙 $N > M(n - 1)$ ，对于任何函数，都存在错误的数据集 S ，使得 S 映射到表中的同一条目。
- 简单运用鸽笼原理就可以证明上述说法。

Carter and Wegman 在1977年提出了哈希函数的通用类的概念。

- 找到一个大于最大可能键值的大素数
 - 令 $\mathbb{Z}_p = \{0, 1, 2, \dots, p - 1\}$ 以及 $\mathbb{Z}_p^* = \{1, 2, \dots, p - 1\}$
- 定义 $h_{ab}(k) = ((ak + b) \bmod p)$ ，则：
 - $\mathcal{H}_{pm} = \{h_{ab} | a \in \mathbb{Z}_p^* \text{ and } b \in \mathbb{Z}_p\}$ 是一个通用哈希族。
- 证明： $Pr_{h \in \mathcal{H}}[h(k) = h(l)] \leq \frac{1}{m}$ 对于所有 $k \neq l$ ，且 $k \in \mathbb{Z}_p$ and $l \in \mathbb{Z}_p$

令 $r = (ak + b) \bmod p$, and $s = (al + b) \bmod p$

- Claim 1: $r \neq s$ (h_{ab} 不会在模 p 上发生碰撞)
 - 证明： $r - s = a(k - l) \bmod p$ ，但是 $a \not\equiv 0 \bmod p$ 并且 $k - l \not\equiv 0 \bmod p$ 。
- Claim 2: 固定 k 和 l ，在 (a, b) 和 (r, s) 之间存在一一对应的映射
- Lemma: $Pr_{h \in \mathcal{H}}[h(k) = h(l)] = Pr_{0 \leq r, s < p}[r \equiv s \bmod m] = Pr[r \equiv s \bmod m]$ (当 (r, s) 是从 $\mathbb{Z}_p$ 中均匀随机选择的
不同值时) $\leq \frac{(\lceil \frac{p}{m} \rceil - 1)}{p - 1} \leq \frac{\frac{p+m-1}{m} - 1}{p - 1} = \frac{(p-1)/m}{p-1} = \frac{1}{m}$

4. Open addressing--开放地址法

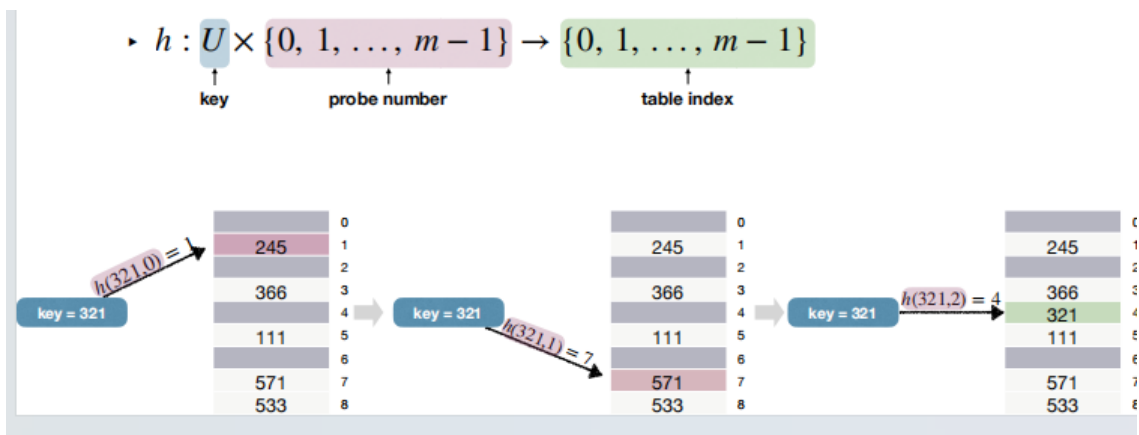
4.1 基本思想

基本思想：没有链接的列表！所有项目存储在表中，每个桶一个项目！

不断进行寻址，直到找到一个空的位置

负载因子： $\alpha = \frac{n}{m} \leq 1$

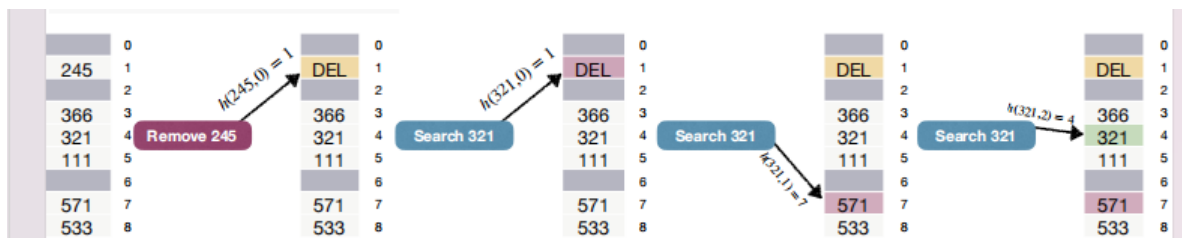
- 哈希函数： $h : U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$



```

1 HashInsert(T, k):
2   i = 0
3   repeat
4     j = h(k, i)
5     if T[j] = NULL or T[j] = DEL
6       T[j] = k
7       return j
8     else i = i + 1
9   until i = m
10  return "overflow"
11
12 HashSearch(T, k):
13  i = 0
14  repeat
15    j = h(k, i)
16    if T[j] = k
17      return j
18    i = i + 1
19  until i = m or T[j] = NULL
20  return NULL
21
22 HashRemove(T, k):
23  pos = HashSearch(T, k)
24  if pos != NULL
25    T[pos] = DEL
26  return pos
27

```



4.2 Linear Probing--线性探测

- $h(k, i) = (h'(k) + i) \bmod m$, 这里 h' 是一个辅助哈希函数。
- 对于 $i = 0, 1, \dots, m-1$ 。给定键 k ，探测的第一个插槽为 $T[h'(k)]$ 。接下来我们探针槽 $T[h'(k) + 1]$ ，依此类推，直到槽 $T[m-1]$ 。然后我们绕到槽 $T[0], T[1], \dots$ ，直到我们最终探测槽 $T[h'(k) - 1]$ 。由于初始探头位置决定了整个探头序列，因此只有 m 个不同的探头序列用于线性探测。
- 线性探测很容易实现，但它存在一个称为主聚类的问题。长时间占用的插槽堆积起来，增加了平均搜索时间。例如，如果我们在表中有 $n = m/2$ 个键，其中每个偶数索引槽都被占用，每个奇数索引槽都是空的，那么平均不成功的搜索需要 1.5 个探测。但是，如果前 $n = m/2$ 个位置是被占用的位置，则平均探针数增加到大约 $n/4 = m/8$ 。可能会出现集群，因为如果一个空插槽前面有 i 个完整的插槽，那么空插槽是下一个填充插槽的概率为 $(i+1)/m$ ，而如果前一个插槽是空的，则概率为 $1/m$ 。因此，占用插槽的运行时间往往会长，并且线性探测不是均匀散列的良好近似值。

4.3 Quadratic Probing--二次探测

- $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$, 这里 h' 是一个辅助哈希函数。
- 如果两个键具有相同的初始探测位置，则它们的探测序列是相同的，因为 $h(k_1, 0) = h(k_2, 0)$ 意味着 $h(k_1, i) = h(k_2, i)$ 。这导致了一种较温和的聚类形式，称为二次聚类。与线性探测一样，初始探针决定了整个序列，因此仅使用 m 个不同的探针序列。

4.4 Double Hashing--双重散列

- $h(k, i) = (h_1(k) + i \cdot h_2(k) \mod m)$, 这里 h_1, h_2 为辅助哈希函数。
- 探测的初始位置为 $T[h_1(k)]$, 连续的探头位置与先前的位置相差 $h_2(k)$, 模 m 。因此, 与线性或二次探测的情况不同, 这里的探针序列以两种方式取决于键 k , 因为初始探针位置、偏移量或两者可能发生变化。对于要搜索的整个哈希表, 值 $h_2(k)$ 必须相对于哈希表大小 m 为基数。否则, 如果 m 和 $h_2(k)$ 对某个键 k 具有最大公约数 $d > 1$, 则搜索键 k 将仅检查哈希表的 $1/d$ 。

4.5 开放地址哈希分析

Theorem 1: 设随机变量 X 为不成功搜索中探测的数量, 则 $E[X] \leq \frac{1}{1-\alpha}$, 其中 $\alpha = \frac{n}{m} < 1$ 。

假设事件 A_i 为: 第 i 个探针指向一个被占用的桶。

- $Pr[A_i | A_1 \cap A_2 \cap \dots \cap A_{i-1}] = \frac{n-(i-1)}{m-(i-1)} \leq \frac{n}{m} = \alpha$
- $Pr[X \geq i] = Pr[A_1 \cap A_2 \cap \dots \cap A_{i-1}] = Pr[A_1] \cdot Pr[A_2 | A_1] \cdot Pr[A_3 | A_1 \cap A_2] \cdots Pr[A_{i-1} | A_1 \cap A_2 \cap \dots \cap A_{i-2}] \leq \alpha^{i-1}$
- $E[X] = \sum_{i=1}^{\infty} Pr[X \geq i] \leq \sum_{i=1}^{\infty} \alpha^{i-1} = 1 + \alpha + \alpha^2 + \dots = \frac{1}{1-\alpha}$

Theorem 2: 假设随机变量 X 为成功搜索中探测的数量, 那么 $E[X] \leq \frac{1}{\alpha} \ln(\frac{1}{1-\alpha})$, 其中 $\alpha = \frac{n}{m} < 1$ 。

设 N_i 为: 搜索第 i 个插入的键时产生的预期探测数。

- 根据前面的分析有 $N_i \leq \frac{1}{1-\frac{i-1}{m}}$
- $E[X] \leq \frac{1}{n} \cdot \sum_{i=1}^n N_i \leq \frac{1}{n} \cdot \sum_{i=1}^n \frac{m}{m-(i-1)} = \frac{m}{n} \cdot \sum_{i=0}^{n-1} \frac{1}{m-i} = \frac{1}{\alpha} \cdot \sum_{k=m-n+1}^m \frac{1}{k} \leq \frac{1}{\alpha} \cdot \int_{m-n}^m \frac{1}{x} dx = \frac{1}{\alpha} \cdot \ln(\frac{n}{m-n}) = \frac{1}{\alpha} \cdot \ln(\frac{1}{1-\alpha})$

4.6 Chaining vs Open-addressing

• Good parts of Open-addressing:

- No memory allocation
 - Chaining needs to allocate list-nodes
- Better cache performance
 - Hash table stores in a continuous region in memory
 - Fewer accesses brings table into cache

• Bad parts of Open-addressing:

- Sensitive to choice of hash functions
 - Clustering is a common problem
- Sensitive to load factor
 - Poor performance when $\alpha \approx 1$